Guia de estudos da Aula 9

Upload de planilhas com Vue 3 e Pinia no CTI Insights

Sumário

1 Objetivo da aula 1

2 Mapa dos arquivos 2

3 Como o Pinia funciona 2

3.1 Ativação no main 2

3.2 Estado getters e actions 2

4 Seleção leitura e tratamento 3

4.1 Entrada do arquivo 3

4.2 Leitura e validação 3

4.3 Correspondência com a aula 3

5 Prévia e sessão 4

6 Comandos para reproduzir 4

7 Roteiro de teste manual 5

8 Erros comuns e soluções 5

9 Backend na próxima etapa 5

10 Checklist de entrega 6

1 Objetivo da aula

A Aula 9 ensina a receber uma planilha, guardar o arquivo no Pinia, tratar as linhas no navegador e exibir uma prévia. Este guia explica a implementação atual do CTI Insights e como conferir os resultados. Backend e banco entram em uma etapa posterior.

Referência: `aula9.pdf`, da professora Tatiana Aparecida de Almeida, SENAI Roberto Mange. Projeto atualizado em 01/10/2026.

2 Mapa dos arquivos

Arquivo em FrontEnd	Responsabilidade
`src/main.js`	Ativa Pinia e roteador e monta a aplicação.
`src/stores/uploadStore.js`	Guarda arquivo, dados, erros e carregamento.
`src/utils/planilha.js`	Lê, padroniza e valida a planilha.
`src/utils/formato.js`	Normaliza textos e interpreta números.
`src/views/UploadView.vue`	Recebe o arquivo e exibe resumo e prévia.
`src/services/README_FUTURO_BACKEND.md`	Planeja a futura integração com Spring Boot.
`tests/uploadStore.test.js`	Testa seleção, processamento, descarte e falha.

Fluxo: **tela → store → leitor da planilha → store → resumo e prévia**. Relatórios e gráficos compartilham o resultado.

3 Como o Pinia funciona

3.1 Ativação no main

`createApp(App)` cria a aplicação; `app.use(createPinia())` registra o Pinia; `app.use(router)` registra as rotas; `app.mount('#app')` monta a interface.

3.2 Estado getters e actions

A aula apresenta blocos `state`, `getters` e `actions`. Nosso projeto utiliza um Setup Store: `ref` e `shallowRef` representam estado; `computed` representa getters; funções representam actions. A organização difere, mas atende ao mesmo fluxo.

Conceito	Exemplos	Papel
Estado	arquivo, dadosOriginais, clientes, erros, carregando	Guarda dados que podem mudar.
Getter	dadosTratados, totalClientes, totalErros, clientesNivelA	Calcula ou expõe dados derivados.
Action	selecionarArquivo, validarArquivo, processarPlanilha, limpar	Executa operações sobre o estado.

`arquivo` usa `shallowRef` e mantém o objeto File em memória. `dadosOriginais` preserva os campos reconhecidos antes da padronização; não copia todas as abas. `dadosTratados` expõe os registros processados, incluindo linhas inválidas identificadas.

`totalClientes` conta somente válidos; `clientesNivelA` conta válidos de nível A; `totalErros` conta ocorrências. Uma linha pode ter mais de um erro.

4 Seleção leitura e tratamento

4.1 Entrada do arquivo

Input e arrastar e soltar chamam `selecionarArquivo`. O store aceita XLSX, XLS e CSV e valida novamente antes de processar. `processarPlanilha()` usa o arquivo guardado e chama `importarPlanilha()`.

Durante a leitura, `carregando` desabilita o processamento e o input. A action encerra esse estado também quando ocorre uma falha. A extensão orienta a entrada, mas não garante que o conteúdo esteja íntegro.

4.2 Leitura e validação

`lerPlanilha()` carrega XLSX sob demanda, lê `arrayBuffer()` e converte a aba em linhas. Prefere `dados_importacao`; na ausência dela, usa a primeira aba. Exige código, empresa, consultor, segmento, nível, faturamento e serviço.

O tratamento remove espaços, reconhece apelidos de segmentos e padroniza níveis. `IND.` pode virar `Indústria`. As listas da aba `listas_referencia`, quando presentes, participam da validação. Cabeçalhos ambíguos são recusados e números inválidos não viram zero.

O período mensal ou anual é preservado sem conversão automática. Faixas numéricas usam o ponto médio, uma estimativa. Faturamento de clientes não representa receita da CTI.

4.3 Correspondência com a aula

Exemplo da professora	Campo interno
codigo_cliente	id
nome_cliente	empresa
nivel_cliente	nivel
dadosTratados	Getter que expõe clientes

Os cabeçalhos aceitos estão em `planilha.js`. Uma planilha com outros nomes pode precisar de adaptação dos apelidos. As validações da base oficial foram preservadas, sem substituir o leitor pelo exemplo simplificado da aula.

5 Prévia e sessão

A tela mostra dez primeiras linhas com linha do Excel, código, cliente, consultor, segmento, nível e validação. A chave da tabela usa a linha, pois códigos duplicados também precisam aparecer para correção. Para consultar todos os registros, abra Relatórios.

O resumo identifica o arquivo processado. Selecionar outro arquivo não substitui o resultado até a nova importação concluir. As linhas inválidas ficam fora dos indicadores.

`sessionStorage` mantém o resultado durante a sessão do navegador. Após F5, o resultado pode ser restaurado, mas o File deve ser selecionado novamente. Descartar planilha limpa arquivo, dados, erros e resultado salvo. Sem importação, as telas de análise usam uma amostra fictícia identificada.

6 Comandos para reproduzir

No PowerShell, na raiz `C:\Users\Instrutor\CTI_SENAI`:

```
git status --short
npm.cmd run dev
```

Saída esperada: Vite informa um endereço local, geralmente `http://localhost:5173/`. Acesse `/app/upload`. A porta pode mudar. Ctrl+C encerra o servidor.

Em outro terminal na mesma raiz:

```
npm.cmd run build
node --test --test-isolation=none FrontEnd/tests/*.test.js
```

Resultado verificado: build termina com `built in ...`, gera `FrontEnd/dist/` e o executor informa **7 testes aprovados, nenhuma falha**. A opção sem isolamento evita o `spawn EPERM` observado neste ambiente. `npm.cmd test` também existe dentro de FrontEnd, mas pode encontrar esse bloqueio.

Os scripts do Vite usam `--configLoader native`, que resolveu a falha de empacotamento da configuração com a dependência nativa do Tailwind. Verificado com Node 24.14.0. Reinicie o servidor após alterar a configuração.

Se o checkout ainda não tiver dependências, execute `npm.cmd run setup` na raiz.

7 Roteiro de teste manual

1. Selecione `Data/CTI_Insights_Base_Validacao_300_Registros.xlsx` em Upload.
2. Clique em Enviar e processar e confira a mensagem de leitura.
3. Confira **300 linhas, 265 válidas, 35 com erro e 37 ocorrências**.
4. Confira a prévia e consulte Relatórios para ver todas as linhas.
5. Arraste um TXT: deve ser recusado antes da leitura.
6. Arraste uma planilha válida e processe pelo mesmo fluxo.
7. Atualize a aba e confira o resultado salvo; selecione novamente para reprocessar.
8. Descarte a planilha e confira a remoção do resumo e da prévia.

Testes automatizados e build passaram. A execução completa deste roteiro no navegador ainda deve ser registrada para comprovar interação e aparência.

8 Erros comuns e soluções

Situação	Causa provável	Solução
npm.ps1 bloqueado	Política do PowerShell	Usar npm.cmd.
spawn EPERM nos testes	Bloqueio de subprocesso	Usar o comando sem isolamento da seção 6.
Falha na configuração do Vite	Carregador padrão	Usar os scripts atuais com configLoader native.
Colunas ausentes	Cabeçalho incompatível	Conferir mensagem e apelidos em planilha.js.
Colunas ambíguas	Duas colunas para um campo	Manter uma coluna por campo reconhecido.
Mais erros que linhas inválidas	Várias ocorrências na mesma linha	Comparar linhas e ocorrências separadamente.
File ausente após F5	File não é restaurado pelo JSON	Selecionar o arquivo novamente.
Cliente fora dos gráficos	Registro inválido	Corrigir o erro indicado na planilha.

9 Backend na próxima etapa

Nesta aula não há API de importação nem gravação no PostgreSQL. Axios não foi instalado. O fluxo futuro será Vue e Pinia → Axios → Spring Boot → PostgreSQL, com implantação no Azure a definir.

O servidor deverá validar novamente os dados, controlar acesso e persistir o resultado. Leia `FrontEnd/src/services/README_FUTURO_BACKEND.md`. Login e contato continuam demonstrativos; sessão do navegador não substitui banco.

10 Checklist de entrega

- ☒ Pinia e XLSX instalados e Pinia ativado.
- ☒ Arquivo guardado no store.
- ☒ Dados originais e tratados disponíveis.
- ☒ Prévia com segmento, nível e validação implementada.
- ☒ README e planejamento do backend atualizados.
- ☒ Sete testes aprovados e build concluído.
- ☐ Conferir seleção e arrastar e soltar no navegador.
- ☐ Registrar evidências da prévia e resumo oficial.
- ☐ Conferir telas em largura ampla e estreita.
- ☐ Acrescentar nome e RA no Word antes da entrega.
- ☐ Revisar git status antes do commit.